

Data Flow Explanation

The `nl_query_service.py` script acts as the central brain of your backend. Using the data flow chart as our guide, here is how the service orchestrates the entire process after a user submits a query from the

React UI.

- 1 **API Endpoint:** The request is first received by the `/api/query` endpoint defined in your `main.py` file. This endpoint's only job is to pass the user's query to the `process_nl_query` function within the service.
- 2 **Intelligent Router:** The first step inside the service is the **Intelligent Router**. This component analyzes the user's question to determine its complexity.
- 3 **Path 1: Simple Query:** If the router decides the question is simple and direct (e.g., "List 5 ships"), it sends the query to the **Direct SQL Chain**. This chain's role is to use an LLM (like the **Gemini API**) to convert the question directly into a single, efficient SQL query, which is then executed against the **PostgreSQL DB**.
- 4 **Path 2: Complex Query:** If the router decides the question is complex or ambiguous (e.g., "Which type of ship was most common in the 19th century?"), it sends the query to the **SQL Agent**. The agent is a more powerful tool that can reason, explore the database schema, and use multiple tools before generating and executing its final SQL query.
- 5 **Narrative Synthesizer:** This is the final step orchestrated by the service. After either Path 1 or Path 2 has returned structured data from the

PostgreSQL DB, the service performs two actions:

- It uses the original query to search the **Vector DB** for relevant unstructured text (the RAG step).
- It sends both the structured data (from SQL) and the unstructured context (from the vector search) to the **Narrative Synthesizer**. This component uses a fast LLM (like

Groq Llama 3) to weave both sources of information into a single, human-readable narrative.

The service then packages this final narrative and the generated SQL into a JSON response to be displayed by the

React UI.